Published in GrabNGoInfo

You have **2** free member-only stories left this month. Sign up for Medium and get an extra one

Amy @GrabNGoInfo Follow

Sep 14, 2022 · 10 min read · · Listen

Save

Time Series Causal Impact Analysis in R

Use Google's R package CausalImpact to do time series intervention causal inference with Bayesian Structural Time Series Model (BSTS)

Open in app ↗ Sign up Sign In

Search Medium

Photo by Immo Wegmann on Unsplash

`CausalImpact` package created by Google estimates the impact of an intervention on a time series. For example, how does a new feature on an application affect the users' time on the app?

In this tutorial, we will discuss using the R package `CausalImpact` to do time series causal inference. You will learn:

- How to set the pre and post periods for the causal impact analysis?
- How to conduct causal inference on time series data?
- How to summarize the causality analysis results and create a report?
- What are the differences between the python and the R packages for `CausalImpact`?

66

Resources for this post:

- Video tutorial for this post on YouTube

- Click here for the Colab notebook.
- More video tutorials on Causal Inference
- More blog posts on Causal Inference
- If you are not a Medium member and want to support me as a writer to keep creating free content (😄 Buy me a cup of coffee ☕), join Medium membership through this link. You will get full access to posts on Medium for $5 per month, and I will receive a portion of it. Thank you for your support!

Let's get started!

Step 1: Install and Import R Libraries

In step 1, we will install and import the R libraries. To learn how to use the statistical package R with Google Colab notebook, please check out my tutorial How to Use R with Google Colab Notebook.

Firstly, let's install `CausalImpact` for time series causal analysis.

```
# Install R version of causal impact
install.packages("CausalImpact")
```

After the installation is completed, we can import the library.

```
# Import library
library(CausalImpact)
```

Step 2： Create Dataset

In step 2, we will create a synthetic time-series dataset for the causal impact analysis. The ground truth of the causal impact is usually not available. The benefit of using a synthetic dataset is that we can validate the accuracy of the model results.

The `CausalImpact` package requires two types of time series:

- A response time series that is directly affected by the intervention.
- And one or more control time series that are not impacted by the intervention.

The idea is to build a time series model to predict the counterfactual outcome. In other words, the model will use the control time series to predict what the response time series outcome would have been if there had been no intervention.

In this example, we created one response time series variable and one control time series variable.

- To make the dataset reproducible, a random seed is set at the beginning of the code.
- Then an autoregressive moving average (ARMA) process is created. The autoregressive (AR) part has two coefficients 0.95 and 0.04, and the moving average (MA) part has two coefficients 0.6 and 0.3.
- After creating the autoregressive moving average (ARMA) process, 500 samples are generated from the process.
- The control time series variable `X` is created by adding a constant value of 10 to the generated values.
- The response time series variable `y` is a function of the control time series variable `X`. It equals 2 times `X` plus a random value.
- The intervention happens at the index of 300, and the true causal impact is 10.

```
# Set up a seed for reproducibility
set.seed(42)

# Create the control ARMA time-series
X <- 10 + arima.sim(n = 500, list(ar = c(0.95, 0.04), ma = c(0.6,
0.3)))

# Create the response time-series
y <- 2*X + rnorm(500)
```

```
# Add the true causal impact
y[301:500] <- y[301:500] + 10
```

A time series usually has a time variable indicating the frequency of the data collected. We created 500 dates beginning on January 1st, 2021 using the `seq.Date` function.

After that, a dataset is created with the control variable `X`, the response varialbe `y`, and the dates as the index.

```
# Create dates
time.points <- seq.Date(as.Date("2021-01-01"), by = 1, length.out =
500)

# Create dataset
df <- zoo(cbind(y, X), time.points)

# Take a look at the data
head(df)
```

Output

```
              y        X
2021-01-01 22.17213 11.37125
2021-01-02 24.08610 11.75323
2021-01-03 23.56082 12.22989
2021-01-04 24.35883 12.27458
2021-01-05 25.18355 12.66373
2021-01-06 26.69157 13.39407
```

Step 3: Set Pre and Post Periods

In step 3, we will set the pre and the post intervention periods.

We can see that The time-series start date is `2021-01-01`, the time-series end date is `2022-05-15`, and the treatment start date is `2021-10-28`.

```
# Print out the time series start date
df[1]
```

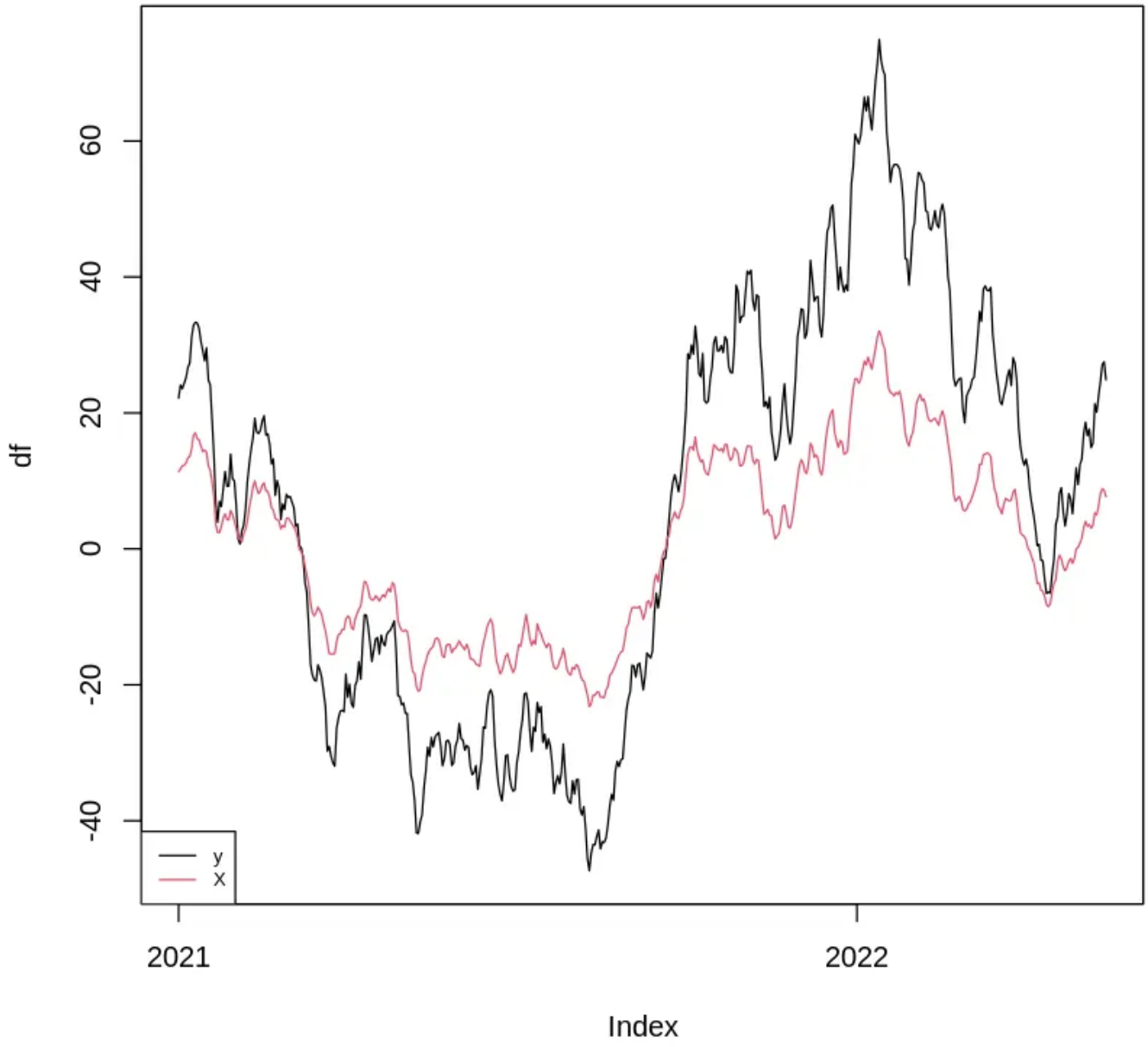
```
# Print out the intervention start date
df[301]

# Print out the time series end date
df[500]
```

Output

```
                 y        X
2021-01-01 22.17213 11.37125
                 y       X
2021-10-28 38.7763 14.5621
                  y        X
2022-05-15 24.88153 7.683928
```

Next, let's visualize the time-series data.

```
plot(df, plot.type="single", col = 1:ncol(df))
legend("bottomleft", colnames(df), col=1:ncol(df), lty=1, cex=.65)
abline(v='2021-10-28', col='red')
```



Causal impact synthetic data visualization — GrabNGoInfo.com

In the chart, the red line is the control time series, the black line is the response time series.

We can see that before the intervention, the response time series has smaller values than the control time series most of the time. After the intervention, the response time series consistently has higher values than the control time series.

The R `CausalImpact` package requires the pre and the post intervention periods to be specified.

In our dataset, the intervention starts on the 301st day, so the pre-intervention time period includes the first 300 days, and the post-intervention time period includes the last 200 days. That corresponds to the start date of `2021-01-01` and end date of `2021-10-27` for the pre-period, and the start date of `2021-10-28` and end date of `2022-05-15` for the post-period.

```
# Set pre-period
pre_period <- c(as.Date("2021-01-01"), as.Date("2021-10-27"))

# Set post-period
post_period <- c(as.Date("2021-10-28"), as.Date("2022-05-15"))
```

Step 4: Raw Differences

In step 4, we will calculate the raw difference between the pre and the post periods.

We can see that the pre-treatment daily average is -11.08, the post-treatment daily average is 32.19, and the raw difference between the pre and the post treatment is 43.27, which is much higher than the true causal impact of 10.

Without causality analysis, we will overestimate the causal impact.

```
# Calculate the pre-daily average
pre_daily_avg <- mean(df$y[1:300])

# Calculate the post-daily average
post_daily_avg <- mean(df$y[301:500])

# Pre-post difference
pre_post_diff = post_daily_avg - pre_daily_avg

# Print out the results
sprintf('The pre-treatment daily average is %f', pre_daily_avg)
sprintf('The post-treatment daily average is %f', post_daily_avg)
sprintf('The raw difference between the pre and the post treatment is
%f', pre_post_diff)
```

Output

```
'The pre-treatment daily average is -11.081051'
'The post-treatment daily average is 32.189605'
'The raw difference between the pre and the post treatment is
43.270656'
```

Step 5: Causal Impact Analysis on Time Series

In step 5, we will execute the causal impact analysis on the time series.

The causality analysis has two assumptions:

- Assumption 1: There are one or more control time series that are highly correlated with the response variable, but not impacted by the intervention. Violation of this assumption can result in wrong conclusions about the existence, the direction, or the magnitude of the treatment effect.
- Assumption 2: The correlation between the control and the response time series is the same for pre and post intervention.

The synthetic time series data we created satisfy the two assumptions.

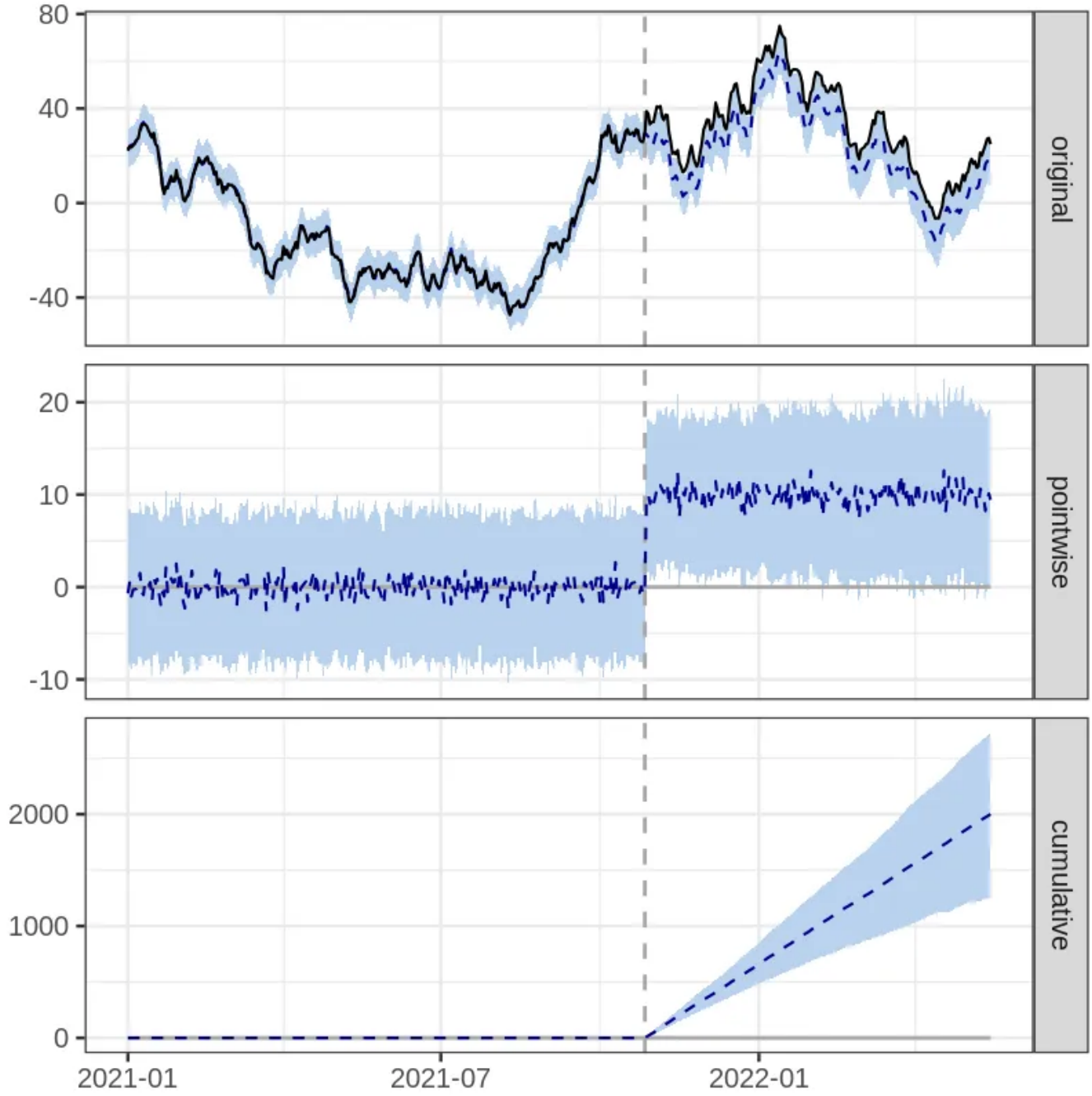
The R `CausalImpact` package has a function called `CausalImpact` that implements a Bayesian Structural Time Series Model (BSTS) on the backend. It has three required inputs:

- `data` takes the dataset name.
- `pre.period` takes the starting and the ending index values for the pre-intervention period.
- `post.period` takes the starting and the ending index values for the post-intervention period.

After saving the output object in a variable called `impact`, we can run `plot(impact)` to visualize the results.

```
# Causal impact model
impact <- CausalImpact(data=df, pre.period=pre_period,
```

```
post.period=post_period)

# Visualization
plot(impact)
```



Causal Impact Analysis on Time Series — GrabNGoInfo.com

The visualization consists of three charts:

- The first chart plots the predicted counterfactual values and the actual values for the post-period.

- The second chart plots the point effects, which are the differences between the actual and predicted values. We can see that the pre-period point effects values are all around 0, and the post-period point effects values are all around the true impact of 10.
- The third chart plots the cumulative effect, which is the cumulative sum of the point effects from the second chart.

Step 6: Time Series Causal Impact Summary

In step 6, we will summarize the causal impact of the intervention for the time series.

The summary from `summary(impact)` tells us that:

- The actual post-intervention average is 32, and the predicted post-intervention average is 22.
- The absolute causal effect is 10, which is the same as the true impact of 10, and much more accurate than the raw difference of 43.27.
- The relative causal effect is 45%.
- The posterior probability of a causal effect is nearly 100%, showing that the model is very confident that the causal impact exists.

```
# Causal impact summary
summary(impact)
```

Output

```
Posterior inference {CausalImpact}

                         Average      Cumulative
Actual                   32           6438
Prediction (s.d.)        22 (1.9)     4438 (381.3)
95% CI                   [19, 26]     [3707, 5184]

Absolute effect (s.d.)   10 (1.9)     2000 (381.3)
95% CI                   [6.3, 14]    [1254.1, 2731]
```

```
Relative effect (s.d.)   45% (8.6%)   45% (8.6%)
95% CI                   [28%, 62%]   [28%, 62%]

Posterior tail-area probability p:   0.00101
Posterior prob. of a causal effect:  99.8993%

For more details, type: summary(impact, "report")
```

We can print the report version of the summary using the `summary(impact, "report")` option.

```
# Causal impact report
summary(impact, "report")
```

Output

```
Analysis report {CausalImpact}

During the post-intervention period, the response variable had an
average value of approx. 32.19. By contrast, in the absence of an
intervention, we would have expected an average response of 22.19. The
95% interval of this counterfactual prediction is [18.54, 25.92].
Subtracting this prediction from the observed response yields an
estimate of the causal effect the intervention had on the response
variable. This effect is 10.00 with a 95% interval of [6.27, 13.65].
For a discussion of the significance of this effect, see below.

Summing up the individual data points during the post-intervention
period (which can only sometimes be meaningfully interpreted), the
response variable had an overall value of 6.44K. By contrast, had the
intervention not taken place, we would have expected a sum of 4.44K.
The 95% interval of this prediction is [3.71K, 5.18K].

The above results are given in terms of absolute numbers. In relative
terms, the response variable showed an increase of +45%. The 95%
interval of this percentage is [+28%, +62%].

This means that the positive effect observed during the intervention
period is statistically significant and unlikely to be due to random
fluctuations. It should be noted, however, that the question of
whether this increase also bears substantive significance can only be
answered by comparing the absolute effect (10.00) to the original goal
of the underlying intervention.
```

```
The probability of obtaining this effect by chance is very small
(Bayesian one-sided tail-area probability p = 0.001). This means the
causal effect can be considered statistically significant.
```

Step 7: CausalImpact Package Differences between Python and R

In step 7, we will talk about the Google `CausalImpact` package differences between Python and R.

The python package was ported from the R package, so most of the time the two packages produce similar results, but sometimes they produce different results.

The differences are caused by assumptions for prior initializations, the optimization process, and the implementation algorithms.

The `pycausalimpact` package documentation highly recommends setting the `prior_level_sd` to `None` when using the Python package, which will let `statsmodel` do the optimization for the prior on the local level component.

The comparison of the results shows that the point estimation values are similar, but the standard deviations are smaller for the estimation with `prior_level_sd` set to `None`. Please see the python version of the tutorial for details.

Step 8: Hyperparameter Tuning for Time Series Causal Impact

To learn how to tune the hyperparameters of the time series causal impact model using the R package CausalImpact, please check out my tutorial Hyperparameter Tuning for Time Series Causal Impact Analysis in R

More tutorials are available on GrabNGoInfo YouTube Channel and GrabNGoInfo.com

Recommended Tutorials

- GrabNGoInfo Machine Learning Tutorials Inventory
- Hyperparameter Tuning for Time Series Causal Impact Analysis in R

- Time Series Causal Impact Analysis in Python
- How to Use R with Google Colab Notebook
- 3 Ways for Multiple Time Series Forecasting Using Prophet in Python
- Four Oversampling And Under-Sampling Methods For Imbalanced Classification Using Python
- Multivariate Time Series Forecasting with Seasonality and Holiday Effect Using Prophet in Python
- Time Series Anomaly Detection Using Prophet in Python
- Autoencoder For Anomaly Detection Using Tensorflow Keras
- Databricks Mount To AWS S3 And Import Data
- Hyperparameter Tuning For XGBoost
- One-Class SVM For Anomaly Detection
- Sentiment Analysis Without Modeling: TextBlob vs. VADER vs. Flair
- Recommendation System: User-Based Collaborative Filtering
- How to detect outliers | Data Science Interview Questions and Answers
- Causal Inference One-to-one Matching on Confounders Using R for Python Users
- Gaussian Mixture Model (GMM) for Anomaly Detection
- Time Series Anomaly Detection Using Prophet in Python

References

- CausalImpact 1.2.1, Brodersen et al., Annals of Applied Statistics (2015). http://google.github.io/CausalImpact/
- R documentation for arima simulation
- pycausalimpact documentation

Join Medium with my referral link - Amy @GrabNGoInfo

Read every story from Amy (and thousands of other writers on Medium). Your membership fee directly supports Amy and...

medium.com

Causal Inference Causality Time Series Forecasting Time Series Analysis

Data Science

Enjoy the read? Reward the writer.[Beta]

Your tip will go to Amy @GrabNGoInfo through a third-party platform of their choice, letting them know you appreciate their story.

Give a tip

Get an email whenever Amy @GrabNGoInfo publishes.

Your email

Subscribe

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play